MyCodeSchool

Cifre: din C++ în Python

Partea 1 · David · teorie

Algoritmii pe cifre îi știi din C++. Aici vezi cum arată aceiași algoritmi în Python, în **două stiluri**: **A — ca în C++** (`while`, `%`, `//`, scrii tu algoritmul) și **B — stil Python** (numărul devine text cu `str(n)` și folosești ce are Python gata făcut). La bac scrii stilul A; în proiecte reale, stilul B.

1. Același program, în ambele limbaje

main.cpp

```cpp
#include <iostream>
using namespace std;

int suma_cifrelor(int n)
{
    int suma = 0;
    while (n > 0)
    {
        suma += n % 10;
        n /= 10;
    }
    return suma;
}

int main()
{
    int n;
    cin >> n;
    cout << suma_cifrelor(n) << endl;
    return 0;
}
```

main.py — stil A

```python
def suma_cifrelor(n: int) -> int:
    suma = 0
    while n > 0:
        suma += n % 10
        n //= 10
    return suma


def main() -> None:
    n = int(input())
    print(suma_cifrelor(n))


if __name__ == "__main__":
    main()
```

C++	Python
`{ ... }` marchează blocul	`:` + **indentare de 4 spații** marchează blocul; fără acolade, fără `;`
`int suma = 0;` — tipul e declarat	`suma = 0` — tipul vine din valoare
`int suma_cifrelor(int n)`	`def suma_cifrelor(n: int) -> int:` — tipurile din semnătură documentează, Python nu le verifică
`cin >> n;`	`n = int(input())` — `input()` întoarce **text**, `int(...)` îl face număr
`cout << a << " " << b << endl;`	`print(a, b)` sau `print(f"{a} {b}")`
`int main()` pornește singur	`main()` e apelat de `if __name__ == "__main__":`

Stil B — aceeași funcție

```python
def suma_cifrelor_py(n: int) -> int:
    return sum(int(c) for c in str(n))
```

2. Operatorii care se schimbă

Expresie	C++ (int)	Python	De ce
`7 / 2`	3	**3.5**	în Python `/` dă mereu număr real
`7 // 2`	—	3	`//` = împărțirea întreagă din C++
`-17 % 10`	-7	**3**	Python rotunjește în jos, spre –∞
`-17 // 10`	-1	**-2**	aceeași regulă
`2 ** 100`	depășire	1267650600228229401496703205376	`int` din Python n-are limită
`nr++`	merge	**SyntaxError**	scrii `nr += 1`
`a && b`, `a || b`, `!a`	merge	`a and b`, `a or b`, `not a`	cuvinte, nu simboluri
`true`, `false`	merge	`True`, `False`	cu majusculă

3. Două rezultate dintr-o funcție

În C++ întorci două valori prin parametri cu `&`. În Python un `int` primit ca parametru nu poate fi schimbat pentru cel care a apelat funcția, așa că întorci **amândouă valorile** deodată, cu `return mn, mx`, și le primești cu `mn, mx = ...`. Tipul e `tuple[int, int]`. Ambele programe afișează `3 8`.

C++ — parametri prin referință

```
void min_max(int n, int &mn, int &mx)
{
    mn = 9;
    mx = 0;
    do
    {
        int c = n % 10;
        if (c < mn) mn = c;
        if (c > mx) mx = c;
        n /= 10;
    } while (n > 0);
}

int mn, mx;
min_max(4738, mn, mx);
cout << mn << " " << mx;
```

Python stil A — return cu două valori

```
def min_max(n: int) -> tuple[int, int]:
    mn = 9
    mx = 0
    while True:
        c = n % 10
        if c < mn:
            mn = c
        if c > mx:
            mx = c
        n //= 10
        if n == 0:
            return mn, mx


mn, mx = min_max(4738)
print(mn, mx)
```

Bucla `do … while` nu există în Python: o scrii ca `while True:` cu ieșire prin `return` (sau `break`) la final — așa se face măcar un pas și pentru `n = 0`.

4. Stilul B: numărul ca text

Cu `s = str(4738)` numărul devine textul `"4738"`, iar cifrele sunt caracterele lui. Pentru `s = "4738"`:

Scrii	Obții	Echivalentul din stilul A
`len(s)`	4	numărul de cifre
`for c in s:`	`"4"`, `"7"`, `"3"`, `"8"`	parcurgerea cifrelor, dar de la stânga
`int(c)`	cifra ca număr	`n % 10`
`s[::-1]`	`"8374"`	oglinditul
`sum(int(c) for c in s)`	22	suma cifrelor
`max(s)`, `min(s)`	`"8"`, `"3"`	cifra maximă / minimă (ca text)
`s.count("7")`	1	de câte ori apare cifra 7
`s.replace("7", "")`	`"438"`	ștergerea cifrei 7
`"".join(c for c in s if int(c) % 2 == 0)`	`"48"`	păstrezi doar cifrele pare
`int("0438")`	438	textul devine iar număr

`sum(int(c) for c in s)` se citește „suma lui `int(c)`, pentru fiecare `c` din `s`”; cu `if` la final păstrezi doar elementele care respectă condiția. Din `math` mai ai `gcd(a, b)` (cmmdc) și `isqrt(n)` (radicalul întreg), după `from math import gcd, isqrt`.

5. Capcane

`/` în loc de `//`: `n = n / 10` face din 1234 numărul 123.4, apoi 12.34… `n` nu ajunge exact la 0 prin tăiere de cifre, iar `numar_cifre(1234)` dă 327 în loc de 4.

Text în loc de număr: fără `int(...)`, `input()` rămâne text: dacă tastezi 5, `input() * 2` dă `"55"`, iar `"12" + "3"` dă `"123"`, nu 15.

Cazul 0: `while n > 0:` nu face niciun pas pentru `n = 0`. Numărul 0 are o cifră: verifici mereu ce întoarce funcția ta pentru 0.

Indentarea: o linie mutată cu un spațiu iese din bloc sau dă `IndentationError`. Folosești mereu 4 spații (VS Code le pune singur la `Tab`).